2026-6-16 C++异常

- 1. 异常的基本概念与行为
  - 1.1 异常的概念
  - 1.2 异常的抛出和捕获
- 2. 异常的执行规则
  - 1.3 栈展开
  - 1.4 查找匹配的处理代码
  - 1.5 异常重新抛出
- 3. 异常的安全与规范
  - 1.6 异常安全问题
  - 1.7 异常规范
- 4. 标准库异常

1. 异常的基本概念与行为

1.1 异常的概念

1. 异常处理机制允许程序中独立开发的部分能够在运行时就就出现的问题进行通信并作出相应的处理，异常使得我们能够将问题的检测与解决问题分开，程序的一部分负责检测问题的出现，然后将解决问题的任务传递给程序的另一个部分，检测是不需要知道问题检测的细节的。
2. C语言对于错误的处理方式是错误码，错误码的本质就是对错误信息进行分类编号，拿到错误码以后还要去查询错误信息。但是异常会抛出一个对象，这个对象可以涵盖更全面的各种信息。

1.2 异常的抛出和捕获

1. 程序出现问题时，我们通过抛出(throw)一个对象来引发一个异常，该对象的类型以及当前的调用链决定了应该由那个catch的处理码来处理该异常。
2. 被选中的处理代码是调用链中与该对象类型匹配且离抛出异常位置最近的那一个。根据抛出对象的类型和内容，程序的抛出异常部分告知异常处理部分到底发生了什么错误。
3. 当throw执行时，throw后面的语句将不再被执行。程序的执行从throw位置跳到与之匹配的catch模块，catch可能是同一函数中的一个局部的catch，也可能是调用链中另一个函数中的catch，控制权从throw位置转移到了catch位置。在这里需要注意的是

> 1.沿着调用链的函数可能提早退出
> 2.一但程序开始执行异常处理程序，沿着调用链创建的对象都将销毁

4. 抛出异常对象后，会生成一个异常对象的拷贝，因为抛出的异常可能是一个局部对象，所以需要生成一个拷贝对象，这个拷贝的对象会在catch子句后销毁。（这里的操作类似于函数的传值返回）

2. 异常的执行规则

1.3 栈展开

1. 抛出异常后，程序暂停当前函数的执行，开始寻找与之匹配的catch子句，首先检查throw是否在try内部，如果在则查找匹配的catch子句，找到与之匹配的catch子句就跳到该地方进行处理。
2. 如果当前函数中没有try/catch子句，或者有try/catch子句但是类型不匹配，则退出当前函数，继续在外层调用函数链中查找，上次查找的catch过程被称为栈展开。
3. 如果到达main函数，依旧没有找到匹配的catch子句，**程序会调用标准库的terminame函数终止程序**。
4. 如果找到匹配的catch子句处理后，catch子句之后的代码会继续执行。

1.4 查找匹配的处理代码

1. 一般情况下抛出对象和catch的类型是完全匹配的，如果有多个类型匹配的，就选择离他位置更近的那个。

   > catch允许接收从非常量向常量的类型转换，也就是支持权限的缩小；允许数组转换成指向数组元素类型的指针，函数被转换成指向函数的指针；**允许派生类向基类类型的转换**，实际中继承体系都是用这样的方式实现的。

2. 如果异常达到main函数依旧没有被捕获，就会终止程序，在日常的开发需求中，不是发生严重错误的情况我们是不期望程序终止的，所以一般main函数中最后都会使用catch(...),它可以捕获任意类型的异常，但是它不会知道异常错误是什么？

1.5 异常重新抛出

3. 异常的安全与规范

1.6 异常安全问题

1.7 异常规范

4. 标准库异常